# Recursion and Backtracking Problems and Solutions

## 1. Print All Permutations of a String/Array

### Approach 1: Using Recursion (Backtracking)

- Description: Swap each element with every other element recursively and print the permutations.
- Time Complexity: O(n!)
- Space Complexity: O(n) (for recursion stack)

Pseudocode (C++):

```
void permute(vector& s, int l, int r) {
if (l == r) {
for (char c   s) cout << c;
cout << endl;
}
else {
for (int i = l; i <= r; i++) {
swap(s[l], s[i]);
permute(s, l + 1, r);
swap(s[l], s[i]);  /Backtrack
}
}
}
```

Dry

Run:

For s = ["A", "B", "C"]:

Initial call: permute(["A", "B", "C"], 0, 2)

Swap A with A → ["A", "B", "C"]

Call permute(["A", "B", "C"], 1, 2)

Swap B with B → ["A", "B", "C"]

Call permute(["A", "B", "C"], 2, 2) → Print: "ABC"

Backtrack and swap B with C → ["A", "C", "B"]

Call permute(["A", "C", "B"], 2, 2) → Print: "ACB"

Continue similarly for other permutations.

- Output:

ABC

ACB

BAC

BCA

CAB

CBA

Approach 2: Using Next Permutation

- Description: Using the next_permutation algorithm to generate the next permutation in lexicographical order.

- Time Complexity: O(n! *

n)

- Space Complexity: O(1)

- Pseudocode (C++):

```
void printPermutations(vector& s) {
sort(s.begin(), s.end());
do {
for (int num  s) cout << num;
cout << endl;
} while (next_permutation(s.begin(), s.end()));
}
```

Dry Run:

For s = [1, 2, 3]:

Sort the array: [1, 2, 3]

Print 123, move to the next permutation [1, 3, 2], print 132, and continue till all permutations are printed.

-

Output:

123

132

213

231

312

321

2. N Queens Problem

Approach 1: Backtracking

- Description: Place queens one by one in each row and check if they are safe to be placed (i.e., no queen can attack another).
- Time Complexity: O(n pow 2)
- Space Complexity: O(n)

Pseudocode (C++):

```
bool isSafe(int board[], int row, int col, int n) {
for (int i = 0; i < row; i++) {
if (board[i] == col or abs(board[i] - col) == row - i) return false;
}
return
```

```
true;
}

void solveNQueensUtil(int board[], int row, int n) {
if (row == n) {
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
if (board[i] == j) cout << "Q ";
else cout << ". ";
}
cout << endl;
}
cout << endl;
return;
}

for (int col = 0; col < n; col++) {
if (isSafe(board, row, col, n)) {
board[row] = col;
solveNQueensUtil(board, row + 1, n);
board[row] = -1;  //
```

Backtrack

}

}

}

void solveNQueens(int n) {

int board[n];

memset(board, -1, sizeof(board));

solveNQueensUtil(board, 0, n);

}

Dry Run:

For n = 4:

Start placing queens row by row.

Check for safe positions.

Print solutions once all queens are placed.

- Output:

Q . . .

. Q .

. Q . .

. . .Q.

. Q

.

Q . . .

. . Q.

. Q . .

3. Sudoku Solver

Approach 1: Backtracking

- Description: Try to fill the board row by row, column by column, and check if the number placement is valid.

- Time Complexity: O(9 pow n), where n is the number of empty cells (9 rows, 9 columns).

- Space Complexity: O(1) (in-place solution)

Pseudocode (C++):

```
bool isSafe(vector>& board, int row, int col, char num) {
for (int i = 0; i < 9; i++) {
if (board[row][i] == num or
board[i][col] == num) return false;
}
int startRow = 3 * (row   ß), startCol = 3 * (col   ß);
for (int i = startRow; i < startRow + 3; i++) {
for (int j = startCol; j < startCol + 3; j++) {
if (board[i][j] == num) return
```

```
false;
}
}
return true;
}

bool solveSudokuUtil(vector>& board) {
for (int row = 0; row < 9; row++) {
for (int col = 0; col < 9; col++) {
if (board[row][col] == '.') {
for (char num = '1'; num <= '9'; num++) {
if (isSafe(board, row, col, num)) {
board[row][col] = num;
if (solveSudokuUtil(board)) return true;
board[row][col] = '.';  /Backtrack
}
}
return false;
}
}
}
return true;
}

void solveSudoku(vector>& board)
```

{

solveSudokuUtil(board);

}

Dry Run:

For an unsolved Sudoku board:

Try to fill in numbers in each empty cell while checking if the current configuration is safe.

Continue recursively until the entire board is solved.

- Output:

The board is solved.

4. M Coloring Problem

Approach 1: Backtracking

- Description: Try to color each vertex such that no two adjacent vertices have the same color.

- Time Complexity: O(m pow n), where m is the number of colors and n is the number of vertices.

- Space Complexity: O(n)

- Pseudocode (C++):

bool isSafe(vector>& graph, vector& color, int v, int c) {

for (int i = 0; i < graph.size(); i++) {

if (graph[v][i] == 1 && color[i] == c) return

```
false;
}
return true;
}

bool graphColoringUtil(vector>& graph, vector& color, int m, int v) {
if (v == graph.size()) return true;

for (int c = 1; c <= m; c++) {
if (isSafe(graph, color, v, c)) {
color[v] = c;
if (graphColoringUtil(graph, color, m, v + 1)) return true;
color[v] = 0;  /Backtrack
}
}
return false;
}

bool graphColoring(vector>& graph, int m) {
vector color(graph.size(), 0);
return graphColoringUtil(graph, color, m, 0);
}
```

Dry Run:

For graph = [[0, 1, 1], [1, 0, 1], [1, 1, 0]] and m =

2:

Try to color vertices with two colors ensuring no two adjacent vertices share the same color.

- Output: True (coloring exists)

5. Rat in a Maze

Approach 1: Backtracking
- Description: Move the rat from start to end, using a recursive backtracking approach.
- Time Complexity: O(4(m*n)), where m and n are the maze dimensions.
- Space Complexity: O(m*n)
Pseudocode (C++):

```
bool isSafe(vector>& maze, int x, int y, vector>& solution) {
return (x >= 0 && y >= 0 && x < maze.size() && y < maze[0].size() &&
maze[x][y] == 1 && solution[x][y] == 0);
}

bool solveMazeUtil(vector>& maze, int x, int y, vector>& solution) {
if (x == maze.size() - 1 && y == maze[0].size() - 1) {
solution[x][y] = 1;
return
```

```
true;
}
if (isSafe(maze, x, y, solution)) {
solution[x][y] = 1;
if (solveMazeUtil(maze, x + 1, y, solution)) return true;
if (solveMazeUtil(maze, x, y + 1, solution)) return true;
solution[x][y] = 0;  /Backtrack
}
return false;
}

bool solveMaze(vector>& maze) {
vector> solution(maze.size(), vector(maze[0].size(), 0));
return solveMazeUtil(maze, 0, 0, solution);
}
```

Dry Run:

For maze = [[1, 0, 0], [1, 1, 0], [0, 1, 1]]:

Start from (0, 0), move through possible paths, and backtrack if needed until reaching the destination.

- Output: True

6. Word Break (Print All Ways)

Approach 1: Dynamic Programming +

Backtracking

- Description: Use dynamic programming to check if the string can be segmented into dictionary words, and then use backtracking to find all ways to break it.
- Time Complexity: O(n pow 2)
- Space Complexity: O(n)

Pseudocode (C++):

```
void wordBreakUtil(string s, vector& wordDict, string current, vector&
result, unordered_map& dp) {
if (s.empty()) {
result.push_back(current);
return;
}
for (int i = 1; i <= s.length(); i++) {
string prefix = s.substr(0, i);
if (find(wordDict.begin(), wordDict.end(), prefix) != wordDict.end()) {
wordBreakUtil(s.substr(i), wordDict, current + (current.empty() ?"" :" ")
+ prefix, result, dp);
}
}
}

vector wordBreak(string s, vector& wordDict) {
vector
```

result;

unordered_map dp;

wordBreakUtil(s, wordDict, "", result, dp);

return result;

}

Dry Run:

For s = "catsanddog" and wordDict = ["cat", "cats", "and", "dog"]:

Try all possible splits using backtracking.

- Output:

bash

Copy code

cats and dog

cat sand dog

8. Subset Sums

Approach: Recursive

- Compute all possible subset sums by recursively including and excluding the current element.

Pseudocode:

void subsetSums(vector& arr, int index, int sum, vector& result) {

if (index == arr.size())

```
{
result.push_back(sum);
return;
}
subsetSums(arr, index + 1, sum + arr[index], result);
subsetSums(arr, index + 1, sum, result);
}
```

Dry Run:
Input: arr = [1, 2, 3]
Steps:

Include 1: [1], Exclude 1: []
Include 2: [1, 2], Exclude 2: [1]
Result: [6, 5, 4, 3, 3, 2, 1, 0]
Output: [6, 5, 4, 3, 3, 2, 1, 0]
Time Complexity: O(2 pow N).
Space Complexity: O(N) (recursion stack).

9. Subset-II

Approach: Recursive (Handle Duplicates)
- Sort the array, skip duplicates while generating

subsets.

Pseudocode:

```
void subsetsWithDup(vector& arr, int index, vector& current, vector>&
result) {
result.push_back(current);
for (int i = index; i < arr.size(); ++i) {
if (i > index && arr[i] == arr[i - 1]) continue;  /Skip duplicates
current.push_back(arr[i]);
subsetsWithDup(arr, i + 1, current, result);
current.pop_back();  /Backtrack
}
}
```

Dry Run:

Input: arr = [1, 2, 2]

Steps:

[1], [1, 2], [1, 2, 2], [2], [2, 2]

Output: [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]

Time Complexity: O(2 pow N).

Space Complexity: O(N).

10. Combination Sum-1

- Approach: Recursive (Allow

Repetition)

- Include the same number multiple times to form combinations.

Pseudocode:

```
void combinationSum(vector& candidates, int target, int index, vector&
current, vector>& result) {
if (target == 0) {
result.push_back(current);
return;
}
if (target < 0 or index == candidates.size()) return;

current.push_back(candidates[index]);
combinationSum(candidates, target - candidates[index], index, current,
result);  /Include
current.pop_back();  /Exclude and move forward
combinationSum(candidates, target, index + 1, current, result);
}
```

Dry Run:

Input: candidates = [2, 3, 6, 7], target = 7

Steps:

[2, 2, 3],

[7]

Output: [[2, 2, 3], [7]]

- Time Complexity: O(2 pow N).
- Space Complexity: O(target /min_element).

11. Combination Sum-2

Approach: Recursive (No Repetition + Handle Duplicates)

- Similar to Combination Sum-1, but skip duplicates.

Pseudocode:

```
void combinationSum2(vector& candidates, int target, int index, vector&
current, vector>& result) {
if (target == 0) {
result.push_back(current);
return;
}
for (int i = index; i < candidates.size(); ++i) {
if (i > index && candidates[i] == candidates[i - 1]) continue;  //Skip
duplicates
if (candidates[i] > target) break;  //No need to proceed
current.push_back(candidates[i]);
combinationSum2(candidates, target - candidates[i], i + 1, current,
```

result);
current.pop_back();  /Backtrack
}
}

Dry Run:
Input: candidates = [10, 1, 2, 7, 6, 5], target = 8

Steps:
[1, 7], [2, 6], [1, 2, 5]

- Output: [[1, 7], [2, 6], [1, 2, 5]]
- Time Complexity: O(2 pow N).
- Space Complexity: O(N).

12. Palindrome Partitioning

- Approach: Recursive Backtracking
- Partition the string and check for palindromes recursively.

Pseudocode:
bool isPalindrome(string& s, int left, int right) {
while (left < right) {
if (s[left++] != s[right--]) return

```
false;
}
return true;
}

void partition(string& s, int index, vector& current, vector>& result) {
if (index == s.size()) {
result.push_back(current);
return;
}

for (int i = index; i < s.size(); ++i) {
if (isPalindrome(s, index, i)) {
current.push_back(s.substr(index, i - index + 1));
partition(s, i + 1, current, result);
current.pop_back();  /Backtrack
}
}
}
```

Dry Run:

Input: s = "aab"

Steps:

["a", "a", "b"], ["aa",